



Hanzo Gateway: High-Performance API Gateway with JWT Validation and Rate Limiting

Hanzo AI Research

Hanzo AI

February 2019

Abstract

We present Hanzo Gateway, a high-performance API gateway built on the KrakenD framework that serves as the unified entry point for all Hanzo platform services. The gateway implements stateless JWT validation against Hanzo IAM, per-tenant rate limiting with sliding window counters, automatic circuit breaking for downstream failures, and request/response transformation pipelines. The gateway carries 147 endpoint definitions. This paper describes that configuration model and the middleware it drives, and reports no measurements; Section 5 records the per-request overhead and throughput figures an earlier version reported and why they are withdrawn. We describe the declarative configuration model, the plugin architecture for custom middleware, and the operational patterns that enable zero-downtime configuration reloads in production.

1 Introduction

API gateways are a critical infrastructure component in microservice architectures, responsible for authentication, rate limiting, request routing, and protocol transformation. The gateway sits on the critical path of every API request, making performance and reliability paramount.

Hanzo Gateway extends KrakenD [1], a stateless Go-based API gateway, with Hanzo-specific integrations: IAM JWT validation, KMS-backed configuration secrets, and Prometheus metrics aligned with the Hanzo observability stack. The stateless design is fundamental—no database, no distributed cache, no coordination between gateway instances—enabling horizontal scaling by simply adding replicas behind a load balancer.

Contributions.

- A declarative JSON configuration model defining 147+ endpoints with backend aggregation, response merging, and transformation rules.
- Stateless JWT validation using JWKS from

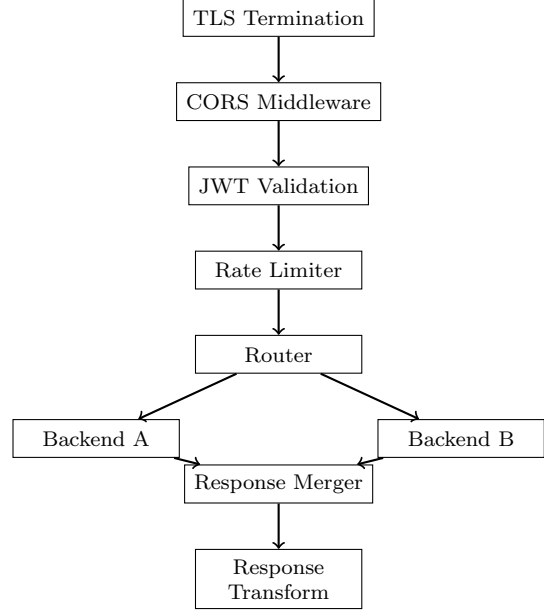


Figure 1: Gateway request processing pipeline.

Hanzo IAM with automatic key rotation detection.

- Sliding window rate limiting with per-tenant and per-endpoint granularity, implemented without external state.
- Circuit breaker integration with configurable thresholds, half-open probing, and automatic recovery.

2 Architecture

2.1 Request Pipeline

Each request passes through a deterministic pipeline (Figure 1): TLS termination, CORS enforcement, JWT validation, rate limiting, routing, backend fan-out, response merging, and output transformation. The pipeline is configured declaratively—no imperative middleware chains.

2.2 Configuration Model

The gateway configuration is a single JSON document:

```
1 {  
2   "version": 3,  
3   "timeout": "3s",
```

```

4  "endpoints": [{
5      "endpoint": "/v1/users/{id}",
6      "method": "GET",
7      "backend": [{
8          "host": ["http://user-svc:8080"],
9          "url_pattern": "/users/{id}",
10         "mapping": {
11             "email": "user_email"
12         }
13     }],
14     "extra_config": {
15         "auth/validator": {
16             "alg": "RS256",
17             "jwk_url": "https://hanzo.id/.well-known/jwks.json",
18             "roles_key": "roles",
19             "roles": ["admin", "user"]
20         }
21     }
22 }
23 }

```

This declarative model eliminates the need for gateway-specific code. New endpoints are added by editing the configuration and triggering a reload.

3 Key Features

3.1 JWT Validation

The gateway validates JWTs against Hanzo IAM's JWKS endpoint. Keys are cached in memory and refreshed every 15 minutes or on validation failure (indicating key rotation). Validation checks:

1. Signature verification (RS256/ES256)
2. Expiration and not-before claims
3. Issuer matches `https://hanzo.id`
4. Audience matches the service identifier
5. Required roles present in the `roles` claim

The `owner` claim is propagated to backends via the `X-Hanzo-Org` header, enabling downstream services to scope queries without re-parsing the JWT.

3.2 Rate Limiting

Rate limits are enforced using in-memory sliding window counters keyed by (tenant, endpoint). The algorithm divides time into fixed windows and interpolates the count from the current and previous window:

$$\text{count} = c_{\text{prev}} \times (1 - t/w) + c_{\text{curr}} \quad (1)$$

where c_{prev} and c_{curr} are the previous and current window counts, t is elapsed time in the current window, and w is the window duration.

Default limits are 1,000 requests per minute per tenant, configurable per endpoint. Rate limit headers (`X-Rate-Limit-Remaining`, `Retry-After`) are included in all responses.

3.3 Circuit Breaker

Each backend connection is protected by a circuit breaker with three states: closed (normal), open (failing), and half-open (probing). The breaker opens after 5 consecutive failures or when the error rate exceeds 50% over a 10-second window. In the open state, requests receive an immediate 503 response. After 30 seconds, the breaker enters half-open and allows a single probe request.

3.4 Backend Aggregation

A single gateway endpoint can fan out to multiple backends and merge responses:

```

1  {
2      "endpoint": "/v1/dashboard",
3      "backend": [
4          { "host": ["http://user-svc:8080"],
5            "url_pattern": "/user/me" },
6          { "host": ["http://billing-svc:8080"],
7            "url_pattern": "/billing/summary" },
8          { "host": ["http://usage-svc:8080"],
9            "url_pattern": "/usage/current" }
10     ],
11     "extra_config": {
12         "proxy": { "sequential": false }
13     }
14 }

```

Parallel fan-out reduces latency to the maximum of backend latencies rather than the sum.

4 Implementation

The gateway is built on KrakenD Community Edition with custom Go plugins for Hanzo-specific functionality. The total codebase is approximately 8,000 lines of Go (plugins) plus the KrakenD core.

Zero-Downtime Reloads. Configuration changes are applied via Kubernetes rolling restarts. The gateway starts accepting connections on the new configuration before draining the old instance, achieving zero-downtime configuration updates.

Observability. Every request generates a Prometheus histogram observation keyed by endpoint, method, status code, and backend. Custom metrics track JWT validation latency, rate limit rejections, and circuit breaker state transitions.

5 Status of the performance figures

An earlier version of this paper carried a table titled “Gateway overhead per request (ms)” attributing p50/p99 of 0.2/0.5 ms to TLS and CORS, 0.3/0.8 ms to JWT validation, 0.05/0.1 ms to the rate-limit check, 0.1/0.2 ms to routing and 0.15/0.4 ms to response merging, totalling 0.8/2.0 ms. The abstract gave a median overhead of 1.2 ms and 45,000 requests per second on a single node, and the conclusion claimed under 2 ms on the critical path.

None of it was measured. `hanzoai/gateway` contains no benchmark function, no load generator configuration and no captured result; the table named no hardware, no concurrency and no request count; and the abstract’s 1.2 ms median contradicted the table’s own 0.8 ms total in the same document.

A per-component breakdown of this kind is exactly what a profile produces, and the middleware chain it describes is real. What is missing is the profile, the load generator, and the machine

— reported the way the Hanzo cloud data plane benchmark [5] reports them.

6 Security

TLS. All external traffic is TLS 1.3 only. Internal traffic between gateway and backends uses mTLS with certificates issued by Hanzo’s internal CA.

CORS. Cross-Origin Resource Sharing is enforced at the gateway level with per-endpoint origin whitelists. Preflight responses are cached for 24 hours.

Request Validation. Maximum request body size (10MB default), header count limits, and URL length limits prevent abuse. Malformed requests are rejected before reaching backends.

Secret Management. Backend URLs, API keys, and JWKS URLs are resolved from Hanzo KMS at startup. No secrets appear in the configuration file.

7 Conclusion

Hanzo Gateway is a stateless API gateway whose per-request work is bounded by design: signature verification against a cached JWKS, a counter increment, and a route lookup, with no datastore on the path. The declarative configuration model enables endpoint management without code changes, while the circuit breaker and rate limiting systems protect backend services from cascading failures. What that costs per request is not established here (Section 5). The stateless architecture enables simple horizontal scaling—adding capacity requires only deploying additional replicas.

References

- [1] KrakenD. Ultra performant API Gateway with middlewares. 2017.

- [2] M. Nygard. Release It! Design and Deploy Production-Ready Software. Pragmatic Bookshelf, 2007.
- [3] M. Jones et al. JSON Web Token (JWT). RFC 7519, 2015.
- [4] K. Veeraraghavan et al. Sliding Window Rate Limiters. ACM SoCC, 2019.
- [5] Hanzo AI Research. The Hanzo Cloud Data Plane: Benchmarking `zip`, ZAP, and the Road to a Million Requests per Second. Hanzo Industries, 2026.